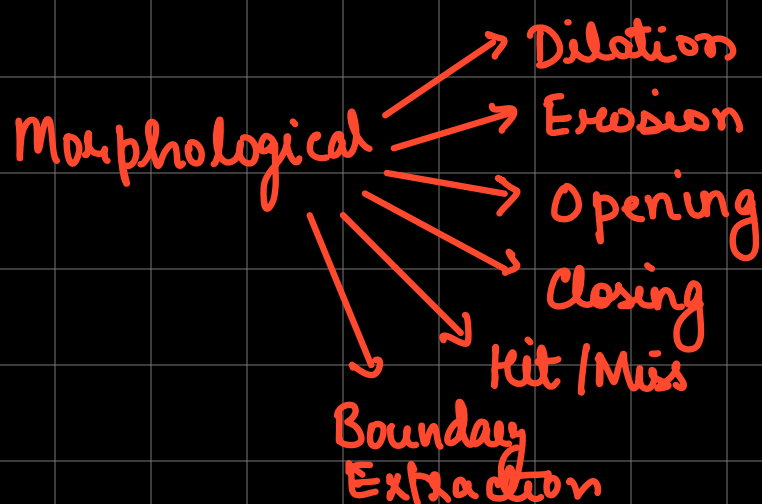


Morphological Operations : process images based on shape using structuring element (kernel).

Commonly used for- noise removal, shape extraction, boundary detection, object separation



⇒ Dilation : structuring element is reflected and shifted left → right, top → bottom, at each shift. In the process we look for overlapping pixel b/w the image & struct el.

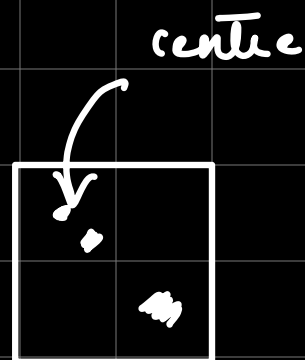
If overlapping then the pixels under the centre position of the se is marked as 1 or

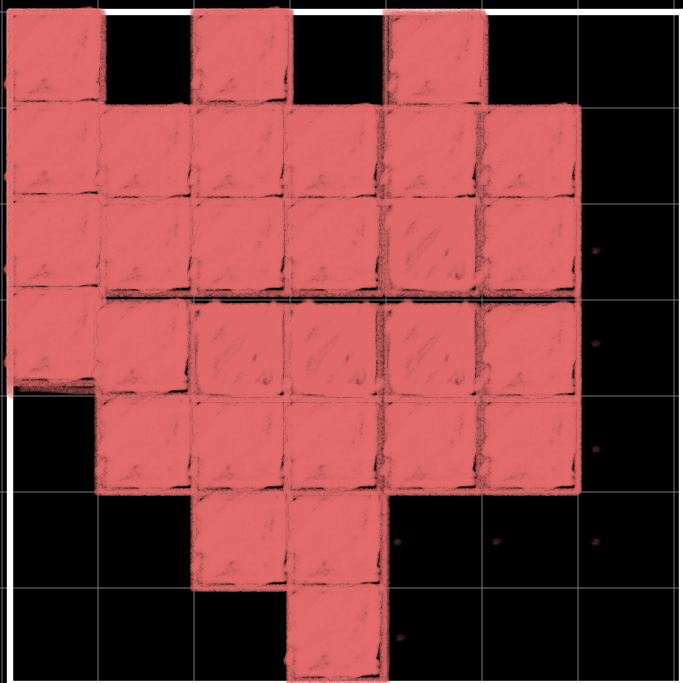
black

$$A \oplus B = \{x \mid (\hat{B})_x \cap A \neq \emptyset\}$$

Appn.  $\rightarrow$  repair breaks  
 $\rightarrow$  repair intrusions  
 $\rightarrow$  enlarge objects

Eg:





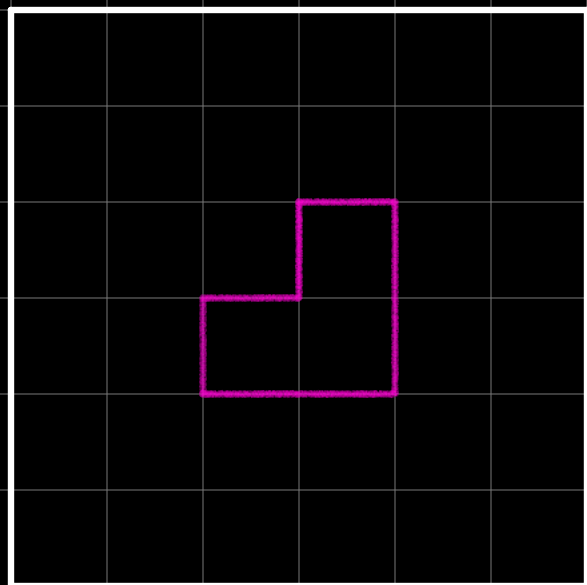
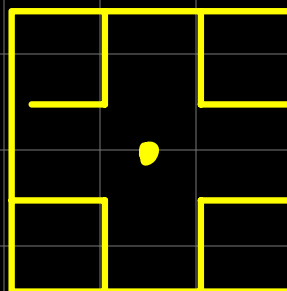
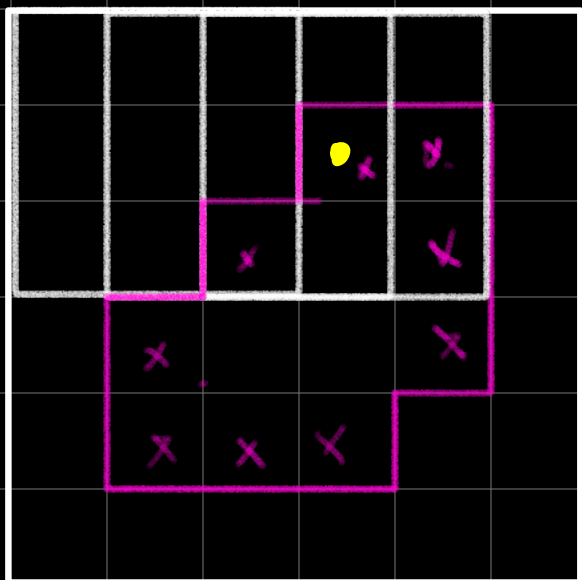
$\Rightarrow$  Erosion : counter process of dilation. Erosion shrinks the image contrary to dilation.

We look for complete overlap with se or not

X complete overlap  $\Rightarrow$  0 at centre of se.  
↳ white

Appn.   
 → Mind   
 → strips away extrusion   
 → strips join objected

$A \rightarrow B$





## Compound Operations

↳ combination of erosion & dilation

- 1> Opening
- 2> Closing

Erosion  $\rightarrow$  Dilation

$\Rightarrow$  Opening

Erosion followed by dilation

$$A \circ B = (A \ominus B) \oplus B$$

Appn.  $\begin{cases} \rightarrow \text{identify gaps} \\ \rightarrow \text{edges become smooth} \\ \rightarrow \text{isolate touching obj.} \end{cases}$

$\Rightarrow$  closing

Dilation followed by erosion

$$A \cdot B = (A \oplus B) \ominus B$$

Appn.  $\rightarrow$  closing gaps

## Hit or Miss Transform

→ used for template matching

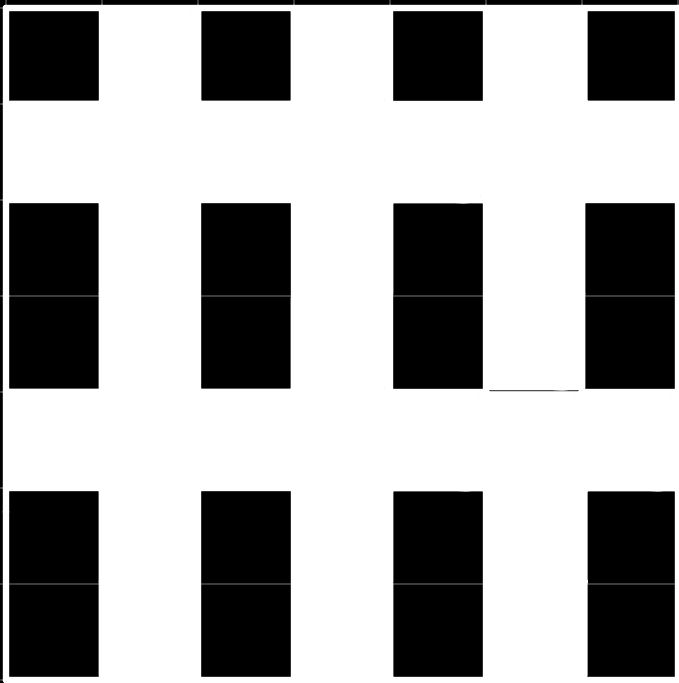
→ 2 templates →  $B$  &  $W-B$  both disjoint

↙ ↘  
match  $\Rightarrow FG$       match  $= BG$

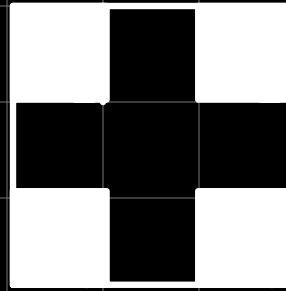
steps involved are as follows:

- 1) Erosion of  $FG$  with  $B$
- 2) Erosion of  $BG$  with  $W-B$
- 3) Intersection of the above steps

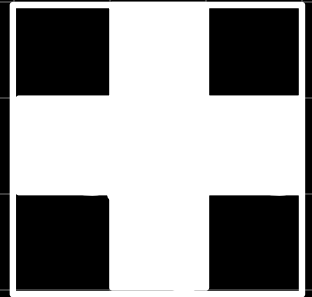
$$HM(X, B) = (X \ominus B) \cap (X^c \ominus (W-B))$$



X



W-B



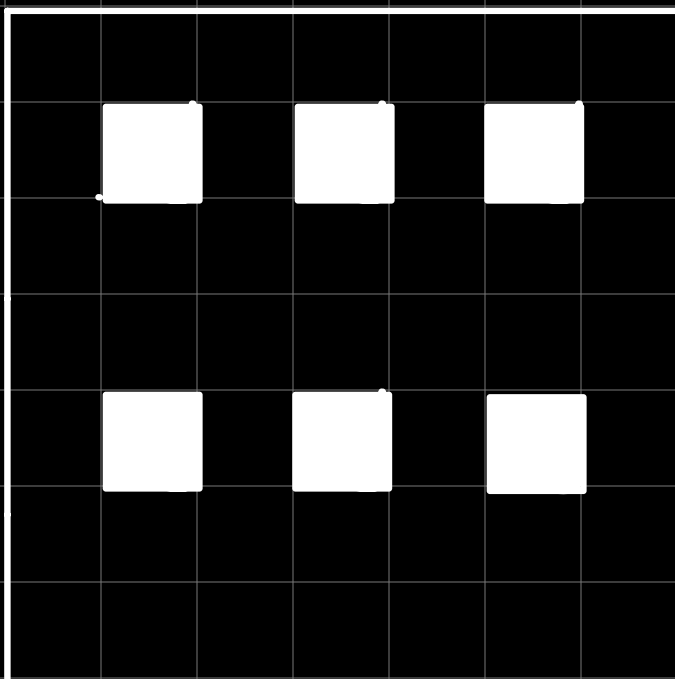
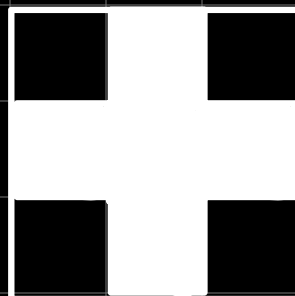
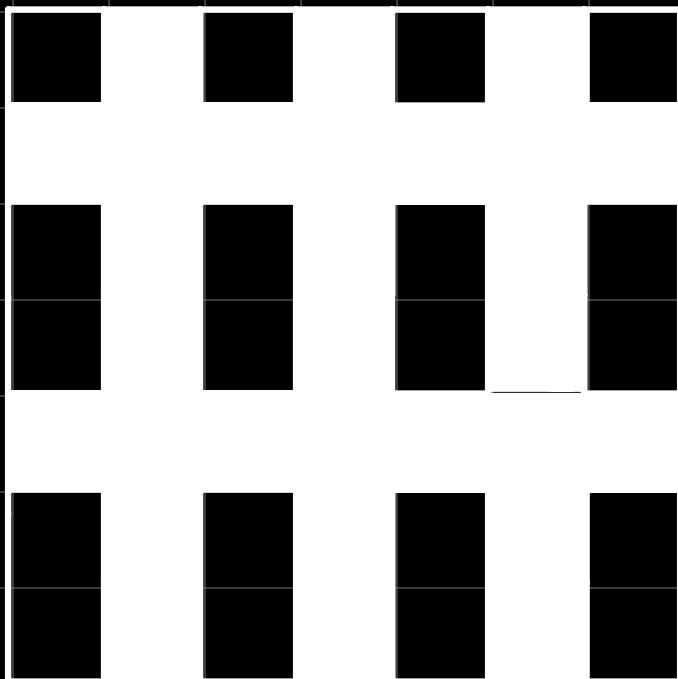
B

$$\Rightarrow HM(X, B) = (X \ominus B) \cup (X^c \ominus (W-B))$$

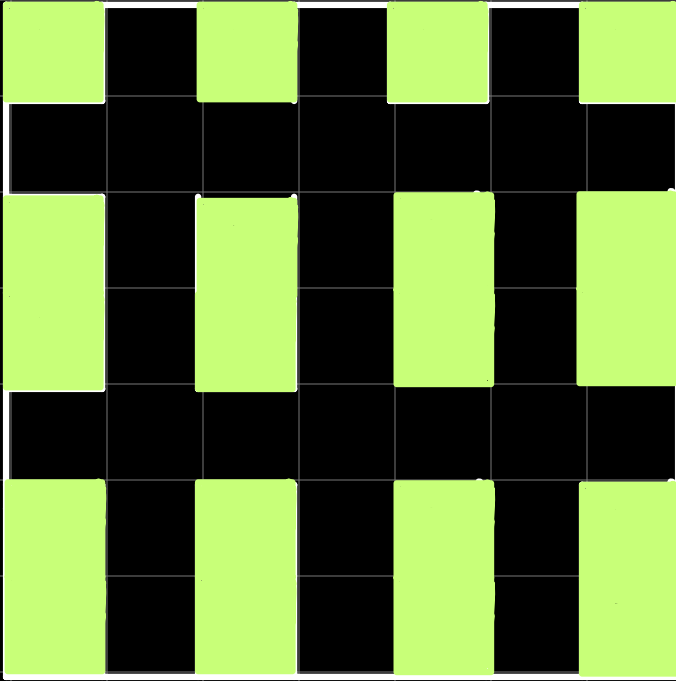
Assume centre is the centre most pixel  
for the structuring element

x

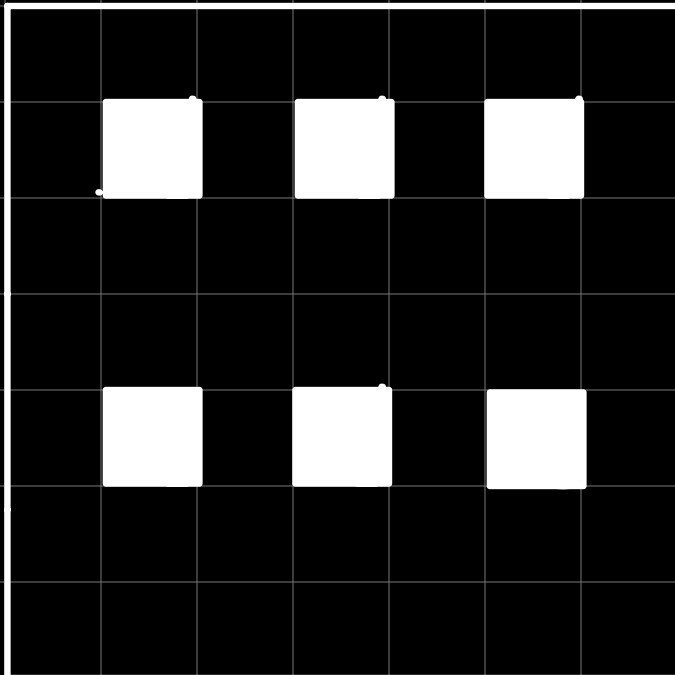
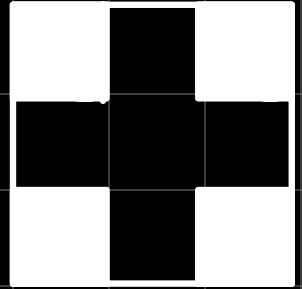
3



$x^c$



$\ominus$



$\Rightarrow$  same ans on intersection

# Image Compression

→ means the reduction of amt of data req. to represent a digital by removing redundant data while retaining necessary info.

→ 2D pixel array → statistically uncorrelated data set.

This is applied prior to storage

At a later time compressed → decompressed to reconstruct og.

$$\text{Compression Ratio} = \frac{\text{Uncompressed Img Size}}{\text{Compressed "}} = \frac{U_{\text{size}}}{C_{\text{size}}}$$

$$U_{\text{size}} = M \times N \times K$$

Q. 8 bit 256x256 pixels

$$C_{size} = 6554 \text{ bytes}$$

$$U_{size} = M \times N \times k$$

$$= 256 \times 256 \times 8 = 65536 \text{ bytes}$$

$$CR = \frac{U_{size}}{C_{size}} = \frac{65536}{6554} = 9.999210$$

Redundancy & CR

$n_1, n_2 \Rightarrow$  carry same info but are diff sizes

$$C_R = \frac{n_1}{n_2}$$

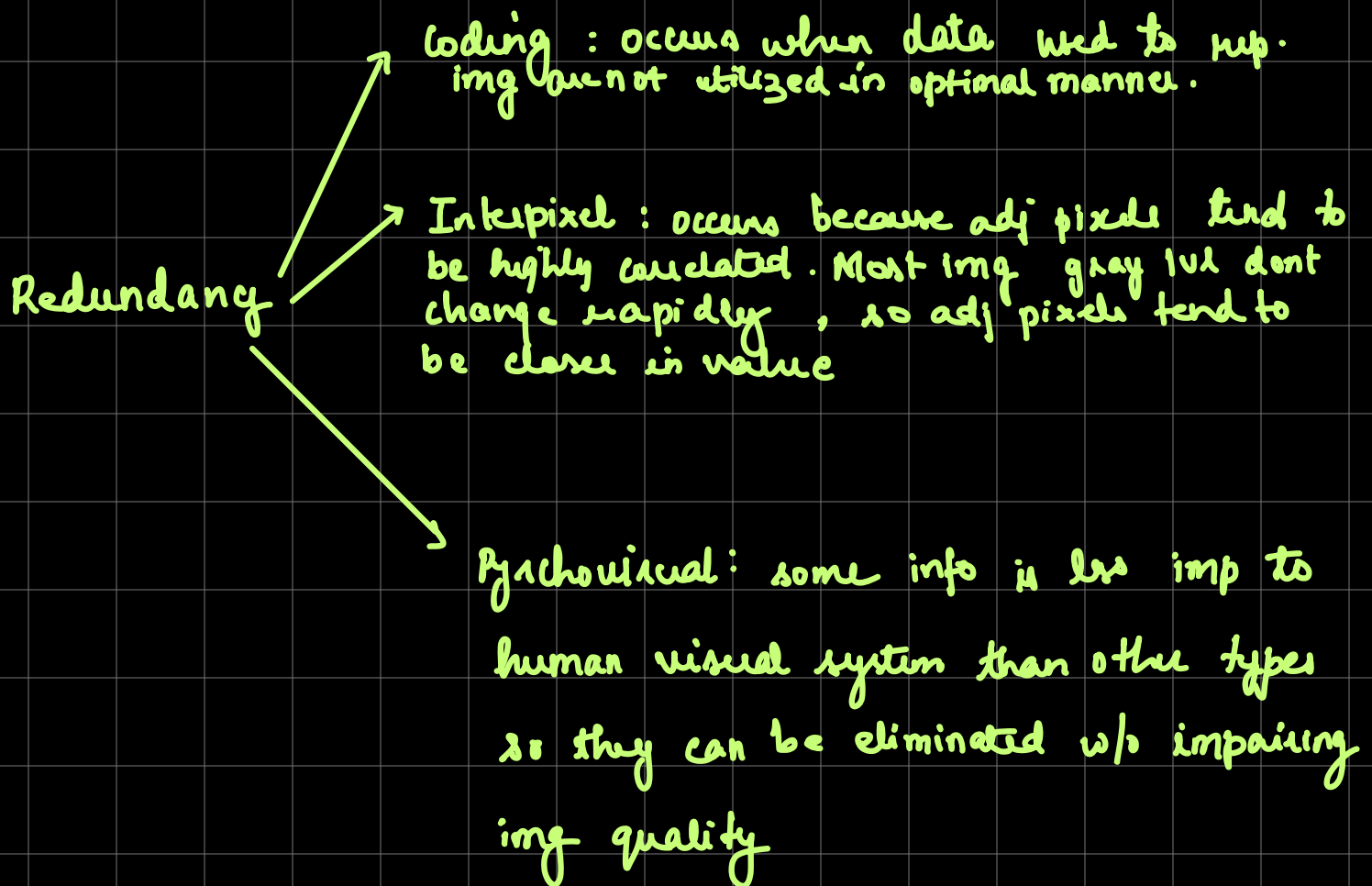
$$R_D = 1 - \frac{1}{C_R}$$

$$n_1 = n_2, \quad C_R = 1 \quad R_D = 0$$



$$n_2 \ll n_1, \quad CR \rightarrow \infty \quad RD \rightarrow 1$$

$$n_2 \gg n_1, \quad CR \rightarrow 0 \quad RD \rightarrow -\infty \quad \text{no compression}$$



Lossless Compression -

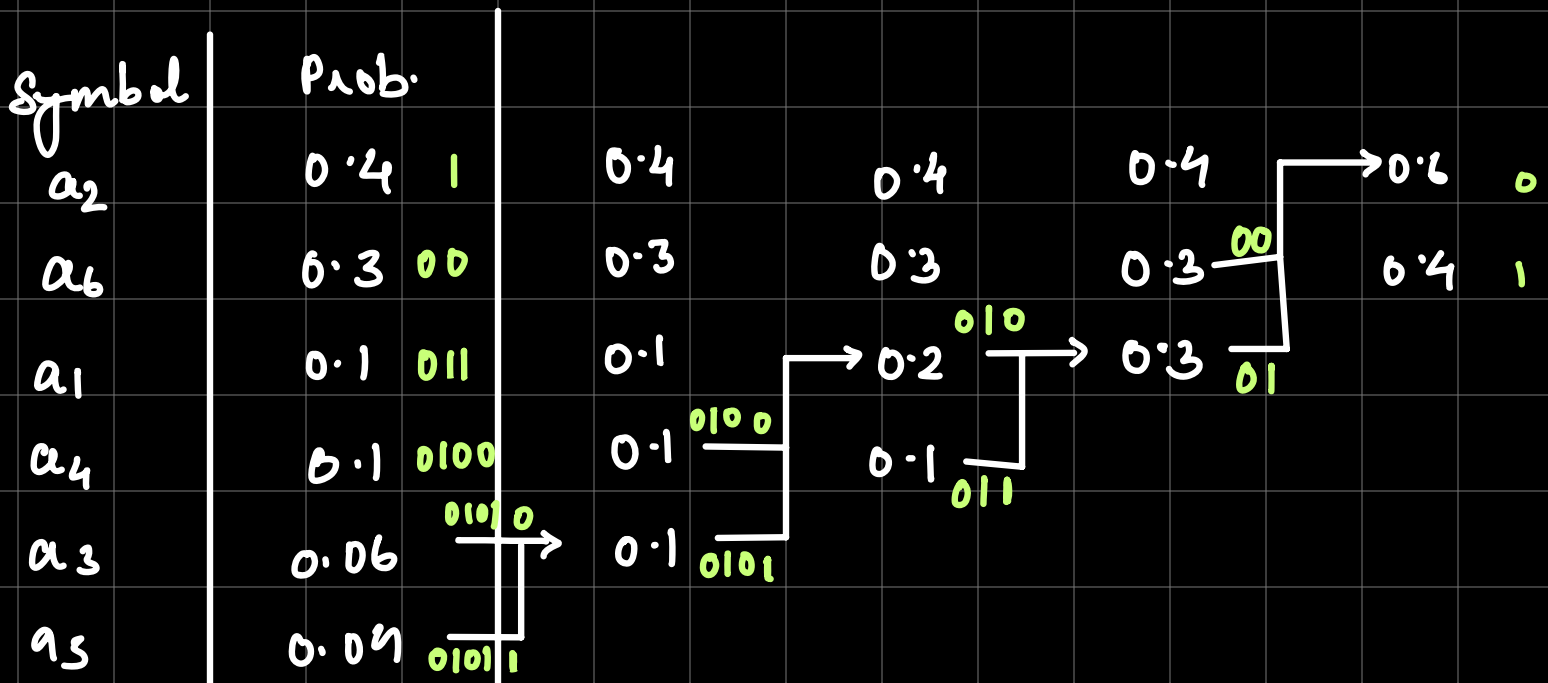
# Huffman Coding

Img size =  $10 \times 10$  (8bit img)

uncompressed size =  $10 \times 10 \times 8 = .500$

|       |    | $P(a_i)$       |  |
|-------|----|----------------|--|
| $a_1$ | 10 | $10/100 = 0.1$ |  |
| $a_2$ | 40 | $40/100 = 0.4$ |  |
| $a_3$ | 6  | $6/100 = 0.06$ |  |
| $a_4$ | 10 | $10/100 = 0.1$ |  |
| $a_5$ | 4  | $4/100 = 0.04$ |  |
| $a_6$ | 30 | $30/100 = 0.3$ |  |

⇒ desc.



$$L_i = \sum p_i L_i$$

$$= 0.4 \times 1 + 0.3 \times 2 + 0.1 \times 3 + 0.1 \times 4 + 0.06 \times 5 + 0.04 \times 5$$

$$= 2.2 \text{ bit/symbol}$$

$$\text{total transmitted} = 10 \times 10 \times 2.2 = 220$$

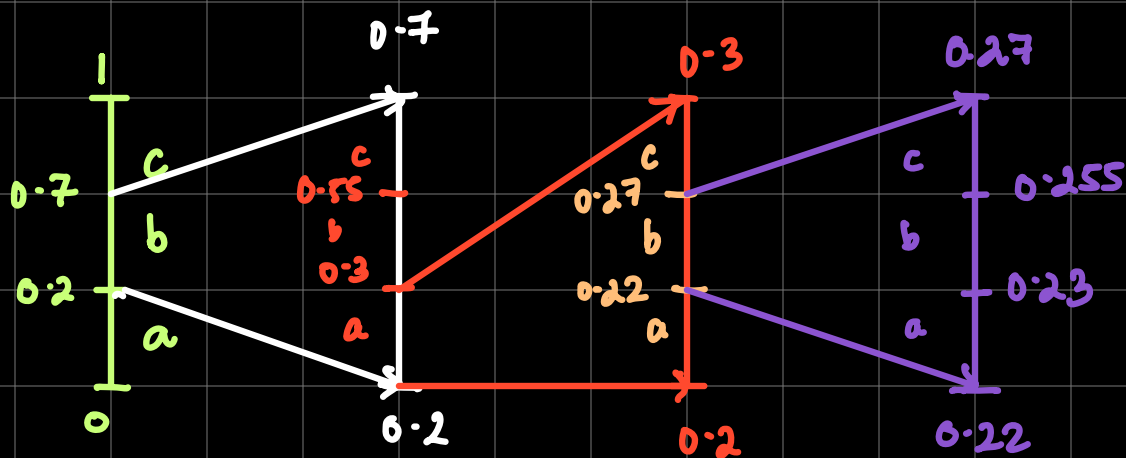
$$\therefore \text{saved} = \frac{500 - 220}{500} = 0.56 = 56\%$$



# Arithmetic coding

- provides one to one mapping
- similar to Huffman but more efficient
- seq. of symbols assigned as per codeword.

Q.  $a=0.2$   $b=0.5$   $c=0.3$  SM =  $babc$



$$d = 0.7 - 0.2 = 0.5$$

$$d = 0.1$$

$$a = 0.2 + 0.5(0.2)$$

$$a = 0.2 + 0.1(0.2)$$

$$b = 0.3 + 0.5(0.5)$$

$$b = 0.22 + 0.1(0.5)$$

$$d = 0.05$$

$$a = 0.22 + 0.05(0.2)$$

$$= 0.23$$

$$b = 0.23 + 0.05(0.5)$$

| Int.           | Sub           |                |                |   |
|----------------|---------------|----------------|----------------|---|
| [0, 1]         | [0 - 0.2]     | [0.2 - 0.7]    | [0.7 - 1.0]    |   |
| [0.2 - 0.7]    | [0.2 - 0.3]   | [0.3 - 0.55]   | [0.55 - 0.7]   | b |
| [0.2 - 0.3]    | [0.2 - 0.22]  | [0.22 - 0.27]  | [0.27 - 0.3]   | a |
| [0.22 - 0.27]  | [0.22 - 0.23] | [0.23 - 0.255] | [0.255 - 0.27] | b |
| [0.255 - 0.27] |               |                |                | c |

# Run Length Encoding

- lossless comp. method where seq. that display redundant data are stored as a single value representing the data block & how many times it appears.
- later during decompression  $\Rightarrow$  img reconstructed exactly
- Works best for simple img/animations with lot of redundant pixels. complex dont have many so RLE makes the file larger

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| a | a | a | b | c | c | c | c |
|   |   |   |   | ↓ |   |   |   |
|   |   | a | 3 | b | 1 | c | 4 |

Loisy

## Vector Quantization

- mapping a vector that has a lot of values to one with smaller no. of values
- image compression  $\Rightarrow$  vector  $\Rightarrow$  small subimage
- works in both spectral / spatial domain
- treats the subimage (vector) as a single entity & quantizes it by reducing total no. of bits req. to represent the subimage
- this is done using a codebook, which stores a fixed set of vectors & then coding the subimage by using the index into the codebook
- better results obtained if we use codebook specifically designed for that img than generic.



→ A. training algo  $\Rightarrow$  vectors  $\Rightarrow$  codebook by finding vectors that best represent blocks in the img

→ Quantization: Reducing gray levels in an img  
ie bits needed to present img.

↙  
even  $\Rightarrow$  Lossy compression

$\Rightarrow$  Quant  $\begin{cases} \text{Uniform} & q(u,v) = \text{constant} \\ & \text{same quant. step everywhere} \\ \text{Non-Uniform} & \text{diff quant steps for} \\ & \text{diff components} \end{cases}$

$\Rightarrow$  Psychovisual redundancy states that to our human visual system certain info are less significant than others

This info is usually neglected by our brain in processing so removal of these info doesn't affect interpretation

⇒ Quant exploits this ⇒ graininess, false contours



IQS (Improved Gray Scale Quant)

⇒ it breaks up edges & contours making it less apparent.

⇒ it adds a pseudo number generated from lower bits of neighbouring pixels to each pixel b4 quant. them.

⇒ MSBs become the coded value.

# Fidelity Criteria

→ Removal of psychovisually red. data  $\Rightarrow$  loss of real info.

→ Fidelity measures how closely the compressed image matches the og.



## Objective

when we can express loss as a fn. of i/p, compressed & decompressed o/p



## Subjective

numeric metric  $\neq$  human perception  
so make humans evaluate side by side the 2 img.

# JPEG (Joint Photographic Experts Group)

→ CR of lossless methods like Huffman, Arithmetic is not high enough for img, & vdo compression

→ JPEG uses transform coding based on 2 observations:

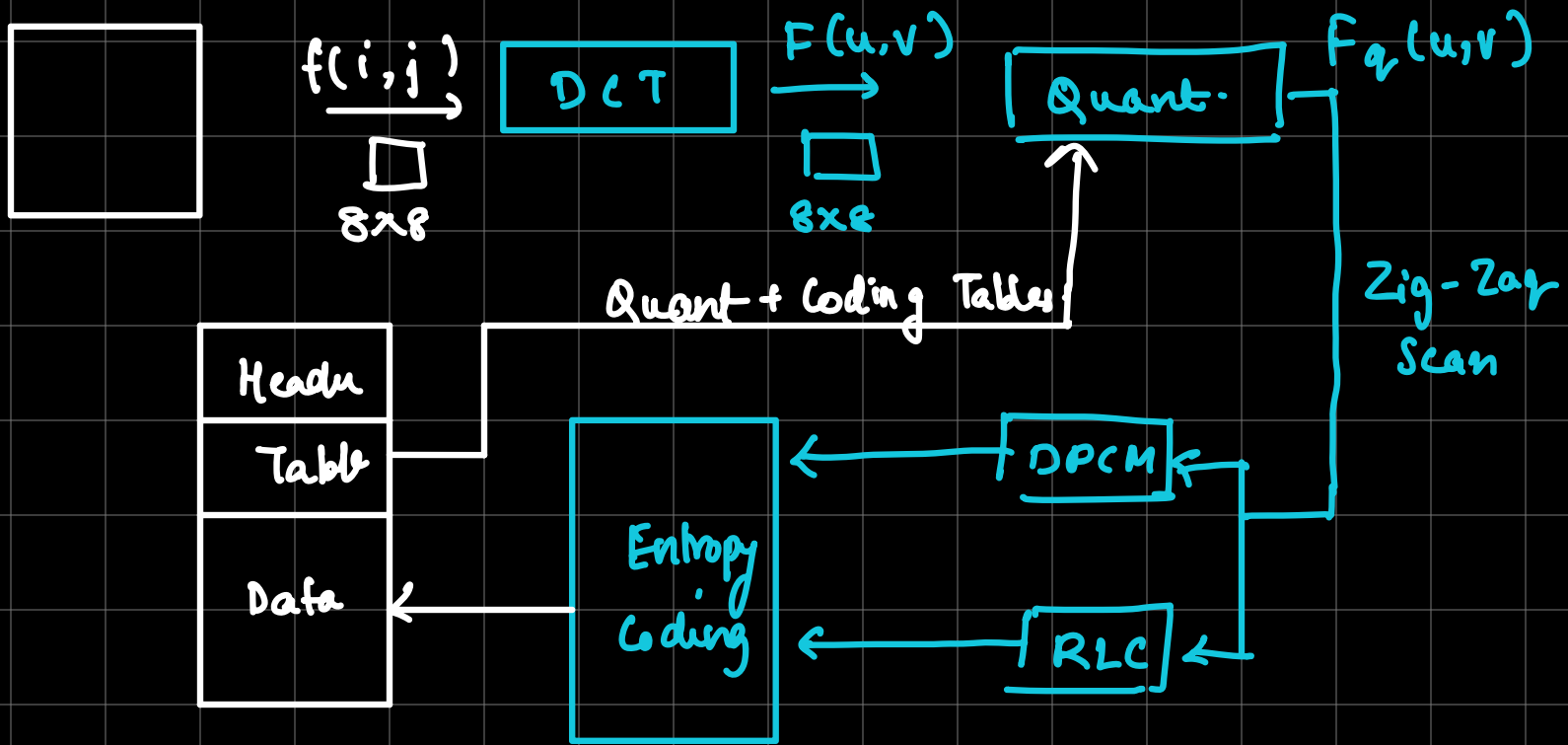
① Most images change smoothly - pixel vals. usually vary gradually.

⇒ In freq. domain this implies low freq

⇒ more relevant info than high freq.

② Humans are more immune to loss of high freq. than low freq. components

→ JPEG Coding



1. Divide img  $8 \times 8$  blocks
2. Apply DCT  $f(i,j) \rightarrow F(u,v)$   
spatial  $\rightarrow$  freq.
3. Quant.
4. DPCM (Differential Pulse Code Modulation)  
 $\Rightarrow$  DC components  
RLC (Run Length Coding)  
 $\Rightarrow$  AC
5. Entropy Huffman  $\Rightarrow$  lossless compression

how image is encoded / stored / transmitted

→ Modes

Sequential

image is encoded  
left → right  
top → bottom  
one scan only

Lossless

compressing  
w/o any info  
loss

⇒ Predictive  
coding rather  
than DCT & Quant.

Progressive

data is transmitted  
in multiple scans  
with gradually increasing  
quality coarse → refined

Hierarchical

transmitting in  
multiple resolutions  
low res → high res

